

Chapter 4. Actor Failure Detection, Recovery, and Self-Healing

In the previous chapters, we covered some of the features of actors and how they relate to handling errors and failure recovery. Let's dig a little deeper into this, shall we?

There are a number of strategies available for handling errors and recovering from failures both at the actor level and at the actor system level.

At the actor level, failure handling and recovery starts with the supervisor-worker relationship. Actors that create other actors are direct supervisors, and for error handling this means that supervisors are notified when a worker runs into a problem. In the supervisor role, there are four well-known recovery steps that may be performed when they are notified of a problem with one of their workers:

1. Ignore the error and let the worker resume processing
2. Restart the worker and perform a worker reset
3. Stop the worker
4. Escalate the problem to the supervising actor of the supervisor

How a supervisor handles problems with a worker is not limited to these four recovery options, but other custom strategies may be used when necessary.

All actors have a supervisor. Actors will form themselves into a hierarchy of worker to supervisor to grand-supervisor and so on (see [Figure 4-1](#)). At the top of the hierarchy is the actor system. If a problem is escalated to the actor system, its default recovery process is to restart the worker (or terminate the worker when more serious problems occur). This supervisory approach frees up the worker from handling its own errors, which means that it is focused completely on performing its tasks. This allows for creating actors with much less error handling code that clutters and hides the main business logic.

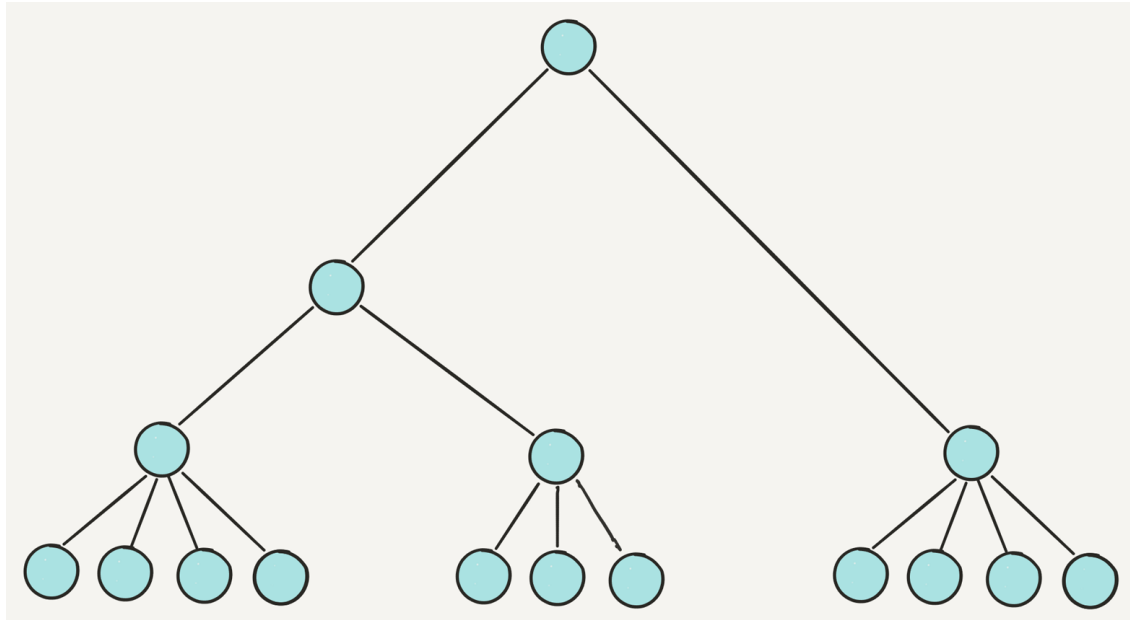


Figure 4-1. Actors form hierarchies

Actors Watching Actors, Watching Actors...

In addition to this supervision strategy, the actor system provides a way for *one actor to monitor another actor*. If the watched actor is terminated, the watcher actor is sent an “actor terminated” message. How the watcher reacts to these terminated messages is up to the design of the watcher actor. This sentinel pattern allows for building some very innovative application features. This pattern is often used to implement forms of self healing into a system (see [Figure 4-2](#)).

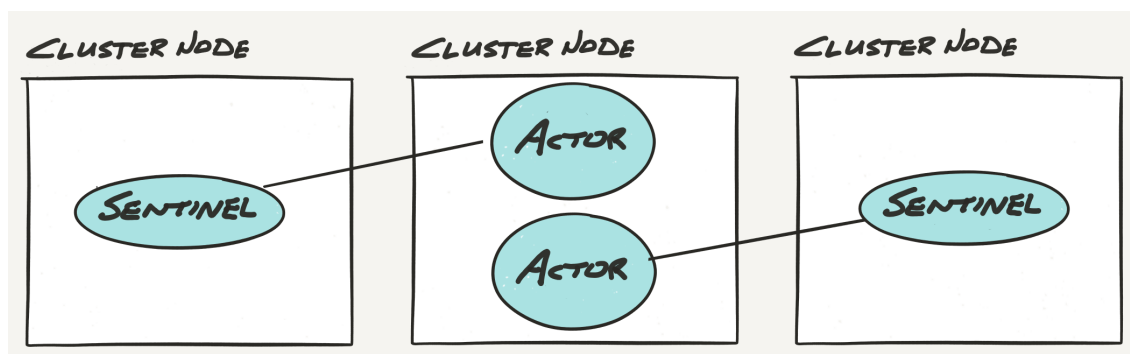


Figure 4-2. Sentinel actors watch actors on other nodes in the cluster

In this example, critical actors may be monitored across nodes in a cluster. If the node where a critical actor is running fails, the *sentinel actors* are notified (see [Figure 4-3](#)). This can trigger some form of recovery and self-healing process by the sentinel actor.

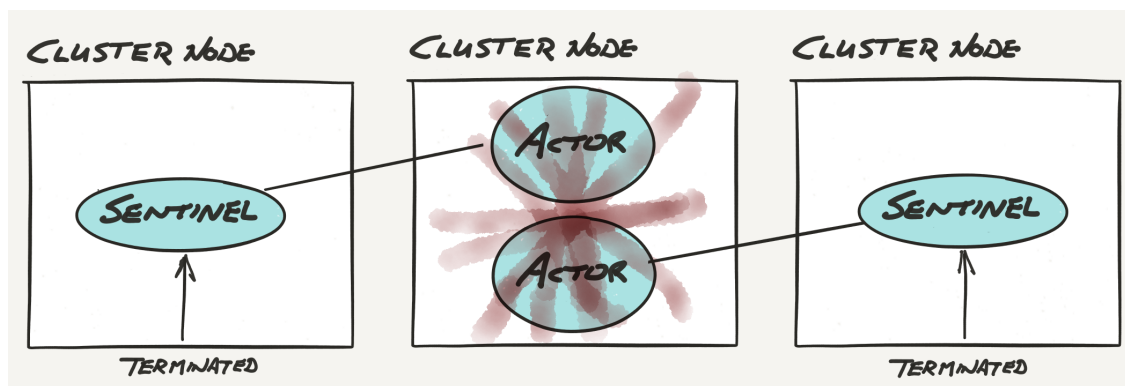


Figure 4-3. When a node fails, the sentinel actors are notified via an actor terminated message

It is common for a set of actors to perform some type of dangerous operation outside of the actor system. By “dangerous operations” we mean one that is more likely to fail from time to time—for example, among a set of actors that perform database operations.

In order to successfully perform these database operations, a lot of things need to be up and running. The backend database server needs to be running and healthy. The network between the actors and the database server needs to be working. When something fails, all of the actors that are trying to do database operations fail to complete their tasks. To exacerbate the problem, in many cases this triggers retries, where either the systems automatically retry failed operations or users seeing errors retry their unsuccessful actions. The end result is that the downed service may be hammered with requests and this increased load may actually hinder the recovery process.

To deal with these types of problems, there is an option to protect vulnerable actors with *circuit breakers* (see [Figure 4-4](#)). Here, a circuit breaker encapsulates actors so that messages must first pass through the circuit breaker, which are generally configured to be in a closed or open state. Normally, the circuit breaker is in a closed state, meaning that the connection allows messages to pass through to the actor. If the actor runs into a problem, the circuit breaker opens the connection and all messages to the wrapped actor are rejected. This stops the flow of requests to the backend service. The idea is to avoid hammering a failed service, such as a down database, when you know that all the requests are going to fail.

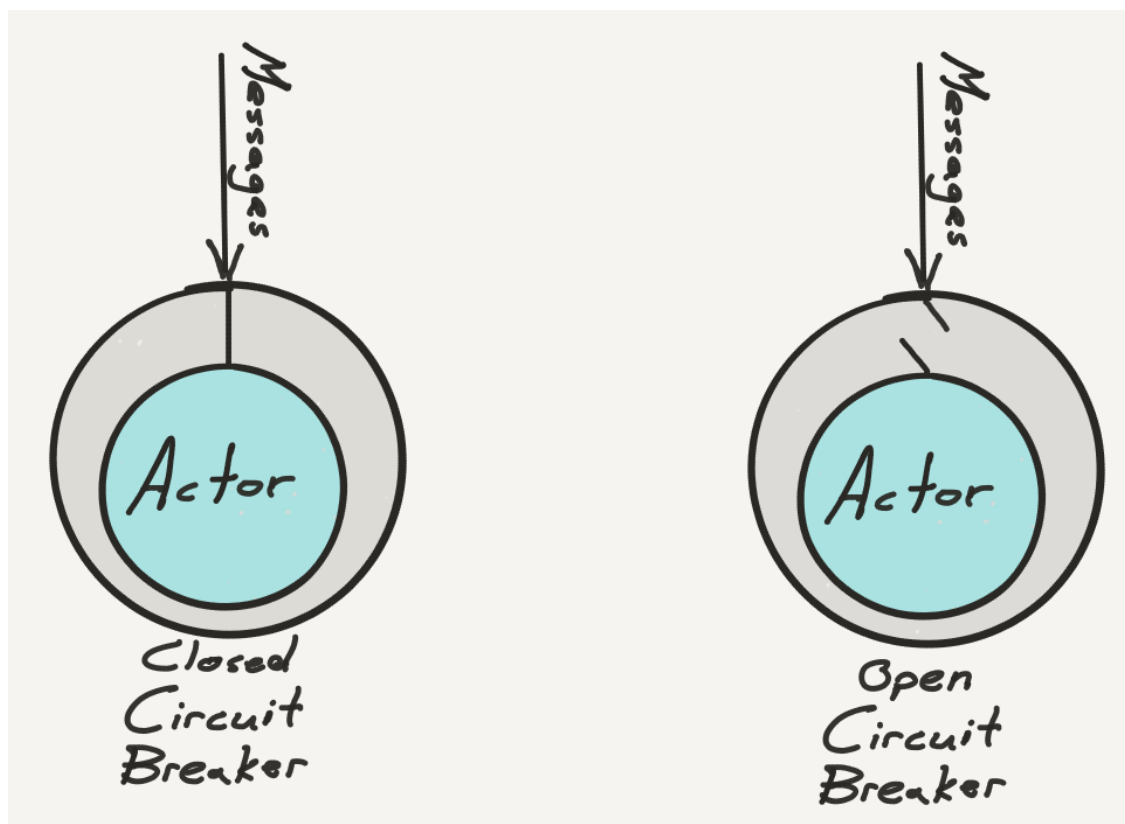


Figure 4-4. Circuit breakers can be used to stop the flow of messages to an actor when something unusual happens

Circuit breakers are configured to periodically allow a single messages to pass to the actor, which is done to allow checks to see if the error has been resolved. If the message fails, the circuit breaker remains open. However, when a message completes successfully the circuit breaker will close, which allows for resuming normal operations. This provides for a straightforward way to quickly ascertain a failure and begin the self-heal process once the problem is resolved. This also comes with the added benefit of providing a way for the system to back off from a failed service.

Another added benefit of the use of circuit breakers is that they provide a way for *avoiding cascading failures*. A common problem that may happen when these types of service failures occur is that the client system may experience a log jam of failing requests. The failed request may generate more retry requests. When the service is down it may take some time before the error is detected due to network request timeouts. This may result in a significant buildup of service requests, which then may result in running out of systems resources, e.g., memory.

On a larger scale, when running a cluster of two or more server nodes, each of the nodes in the cluster monitors the other nodes in the cluster. The cluster nodes are constantly gossiping behind the scenes in order to keep track of each other, so that when a node in the cluster runs into a

problem and fails or is cut off from the other nodes due to a network issue, the remaining nodes in the cluster quickly detect the problem.

Actor flexibility extends even into being notified when there are node changes to the cluster. This not only includes nodes leaving the cluster, but also nodes joining the cluster. This feature allows for the creation of actors that are capable of reacting to cluster changes. Actors that want to be notified of cluster changes register their interest with the actor system. When cluster node changes occur, the registered interested actors are sent a message that indicates what happened. What these actors do when notified is application specific. As an example, actors that monitor state changes to the cluster may be implemented to coordinate the distribution of other actors across the cluster. When a node is added to the cluster, the actors that are notified of the change react by triggering the migration of existing actors to the new node. Conversely, when nodes leave the cluster, these actors react to the failure by recovering the actors that were running on the failed node on the remaining nodes in the cluster.

The main takeaways of this chapter are:

- Actor supervision handles workers that run into trouble, handling error recovery that frees workers to focus on the task at hand.
- Actors may watch for the termination of other actors and react appropriately when this happens.
- Actors may be wrapped in a circuit breaker that can stop the flow of messages to an actor that is unable to perform tasks due to some other, possibly external, problem. Circuit breakers allow for graceful recovery and self-healing, stemming the flow of traffic to a failed service to accelerate the service recovery process.
- Actors may be cluster aware and designed to be notified when nodes join or leave the cluster. This can be used to react to the cluster changes.

